# Percabangan, Perulangan, dan Operasi Git Lanjutan pada Dart

Mata Kuliah: Pemrograman Mobile

Pertemuan: 3

# Agenda Pertemuan Kali Ini

### Kontrol Alur Program pada Dart

Percabangan (if-else, switch-case) dan perulangan (for, while, do-while, for-in)

### Operasi Git Lanjutan

Memperdalam pengetahuan tentang pull, push, branching, merging, dan clone

### Praktik Pemrograman

Mengimplementasikan program Dart dengan kombinasi percabangan dan perulangan

Pertemuan ini akan memperkuat logika pemrograman dan kemampuan kolaborasi kita dalam pengembangan aplikasi mobile.

# Capaian Pembelajaran

**SUB-CPMK3:** Mahasiswa mampu menerapkan pemikiran logis dan kritis dalam mengembangkan aplikasi berbasis mobile sesuai dengan berbagai macam studi kasus yang diberikan.

Setelah mengikuti pertemuan ini, mahasiswa diharapkan dapat:

- Menerapkan struktur percabangan dan perulangan dalam program Dart
- Mempraktikkan operasi Git lanjutan untuk manajemen proyek
- Mengkombinasikan konsep percabangan dan perulangan untuk menyelesaikan masalah

# Apa itu Kontrol Alur?

Kontrol alur adalah mekanisme untuk menentukan **urutan eksekusi instruksi** dalam program.

Tanpa kontrol alur, program akan berjalan secara linear dari atas ke bawah.

Dengan kontrol alur, program dapat:

- Membuat keputusan berdasarkan kondisi tertentu
- Mengulang blok kode beberapa kali
- Melompat ke bagian kode lain

# Jenis Kontrol Alur pada Dart

### Percabangan (Conditional)

- if-else
- else-if
- switch-case

Memilih alur berdasarkan kondisi tertentu

### Perulangan (Looping)

- for
- while
- do-while
- for-in

Mengulang eksekusi blok kode

Kita akan mempelajari masing-masing kontrol alur ini dan penggunaannya dalam pemrograman Dart.

# Percabangan: if Statement

if statement digunakan untuk mengeksekusi blok kode **jika dan hanya jika** suatu kondisi bernilai **true**.

```
int nilai = 75;if (nilai >= 70) {  print('Lulus');}// Output: Lulus
```

Jika kondisi bernilai **false**, blok kode di dalam kurung kurawal akan dilewati dan program melanjutkan eksekusi kode selanjutnya.

# Percabangan: if-else Statement

if-else statement memungkinkan program untuk memilih antara dua blok kode berdasarkan kondisi.

```
int nilai = 65;if (nilai >= 70) {  print('Lulus');} else {  print('Tidak Lulus');}// Output: Tidak Lulus
```

### Evaluasi Kondisi

nilai >= 70 dievaluasi

### Percabangan

Jika true → blok if

Jika false → blok else

### Hasil

Eksekusi salah satu blok kode

# Percabangan: else-if Statement

Digunakan untuk memeriksa beberapa kondisi secara berurutan. Berguna untuk kasus dengan lebih dari dua kemungkinan hasil.

```
int nilai = 85;if (nilai >= 90) {  print('Nilai A');} else if (nilai >= 80) {  print('Nilai B');} else if (nilai >= 70) {  print('Nilai C');} else {  print('Nilai D');}// Output: Nilai B
```

Kondisi diperiksa dari atas ke bawah. Begitu kondisi bernilai **true**, blok kode terkait dieksekusi dan kondisi lainnya **dilewati**.

# Logika Percabangan Bertingkat

Alur program akan memeriksa kondisi satu per satu. Begitu menemukan kondisi yang terpenuhi (true), blok kode terkait akan dieksekusi dan program keluar dari struktur percabangan.

Jika tidak ada kondisi yang terpenuhi, blok else akan dieksekusi sebagai "jalan terakhir".

# Percabangan: switch-case

Digunakan untuk memilih satu dari banyak blok kode berdasarkan nilai tunggal (bukan ekspresi kondisional).

```
String hari = 'Senin';switch (hari) {  case 'Senin':
print('Hari kerja');    break;  case 'Sabtu':  case 'Minggu':
// Multiple cases    print('Hari libur');    break;  default:
print('Hari tidak dikenal');}// Output: Hari kerja
```

**Catatan Penting:**

Wajib ada break; di setiap case

- Multiple cases bisa digunakan untuk hasil yang sama

default dieksekusi jika tidak ada case yang cocok

- Lebih rapi untuk banyak kondisi diskrit dibanding else-if bertingkat

# Kapan Menggunakan if-else vs switch-case?

### Gunakan if-else ketika:

- Kondisi melibatkan perbandingan range (>, <, >=, <=)
- Kondisi kompleks dengan operator logika (&&, ||)
- Jumlah kondisi sedikit (1-3 kondisi)

### Gunakan switch-case ketika:

- Memeriksa nilai tunggal yang sama (==)
- Banyak kondisi dengan nilai diskrit
- Kode lebih mudah dibaca dengan pengelompokan case

Pemilihan yang tepat akan membuat kode lebih efisien dan mudah dibaca!

# Perulangan: for Loop

for loop digunakan untuk mengulang blok kode dengan jumlah iterasi yang telah ditentukan.

```
for (int i = 0; i < 5; i++) {  print('Iterasi ke-$i');}// Output:// Iterasi ke-0// Iterasi ke-1// Iterasi ke-2// Iterasi ke-3// Iterasi ke-4
```

**Sintaks:** for (inisialisasi; kondisi; increment/decrement)

**Inisialisasi:** Dijalankan sekali di awal

**Kondisi:** Diperiksa sebelum setiap iterasi

**Increment/Decrement:** Dijalankan setelah setiap iterasi

Loop berhenti ketika kondisi bernilai **false**.

# Perulangan: while Loop

while loop mengulang blok kode selama kondisi tertentu bernilai **true**. Kondisi diperiksa **sebelum** eksekusi blok kode.

```
int i = 0;while (i < 3) {  print('Angka: $i');  i++;}// Output:// Angka: 0// Angka: 1// Angka: 2
```

Jika kondisi awal bernilai **false**, blok kode tidak akan dieksekusi sama sekali.

# Perulangan: do-while Loop

do-while loop mirip dengan while loop, tetapi kondisi diperiksa **setelah** eksekusi blok kode. Ini menjamin blok kode dieksekusi setidaknya sekali.

```
int i = 0;do {  print('Angka (do-while): $i');  i++;}
while (i < 3);// Output:// Angka (do-while): 0//
Angka (do-while): 1// Angka (do-while): 2
```

Bahkan jika kondisi awal **false**, blok kode tetap dieksekusi sekali:

```
int j = 5;do {  print('Ini dieksekusi');
j++;} while (j < 3);// Output:// Ini
dieksekusi
```

Meskipun j > 3 (kondisi false), blok kode tetap dieksekusi sekali.

# Perbedaan while vs do-while

### while loop

Periksa kondisi **sebelum** eksekusi

Mungkin tidak pernah mengeksekusi blok kode jika kondisi awal false

### do-while loop

Eksekusi blok kode terlebih dahulu, baru periksa kondisi

Selalu mengeksekusi blok kode minimal satu kali

Pilih loop yang tepat berdasarkan kebutuhan program Anda. Jika blok kode harus dieksekusi setidaknya sekali, gunakan do-while.

# Perulangan: for-in Loop

for-in loop digunakan untuk mengiterasi elemen-elemen dalam koleksi seperti List, Set, atau Map. Lebih sederhana daripada for loop tradisional ketika bekerja dengan koleksi.

```
List buah = ['Apel', 'Jeruk', 'Mangga'];for (String b in buah) {  print('Saya suka $b');}// Output:// Saya suka Apel// Saya suka Jeruk// Saya suka Mangga
```

Keunggulan for-in:

- Sintaks lebih sederhana dan mudah dibaca
- Tidak perlu mengakses elemen menggunakan indeks
- Tidak perlu mengetahui panjang koleksi

# Perbandingan Jenis Perulangan

| Jenis Loop | Kapan Menggunakan | Kelebihan |
|---|---|---|
| for | Ketika jumlah iterasi diketahui | Kontrol presisi, variabel indeks tersedia |
| while | Ketika iterasi bergantung pada kondisi yang tidak pasti | Fleksibel, baik untuk jumlah iterasi yang tidak diketahui |
| do-while | Ketika blok kode harus dieksekusi minimal sekali | Menjamin eksekusi minimal satu kali |
| for-in | Ketika iterasi seluruh elemen koleksi | Sintaks sederhana, tidak perlu indeks |

Pemilihan jenis perulangan yang tepat akan membuat kode lebih efisien dan mudah dipahami.

# Studi Kasus: Percabangan dan Perulangan

Program untuk menampilkan tabel perkalian dengan format yang berbeda berdasarkan nilai.

```
void main() { int angka = 7;  if (angka <= 5) { // Format sederhana untuk angka kecil for (int i = 1;
i <= 10; i++) { print('$angka x $i = ${angka * i}'); } } else { // Format tabel untuk angka besar
print('Tabel perkalian $angka:'); print('-------------------'); for (int i = 1; i <= 10; i++) {
String hasil = '$angka x $i = ${angka * i}'; // Tambahkan highlight untuk hasil yang habis dibagi 2
if ((angka * i) % 2 == 0) { print('$hasil (genap)'); } else { print(hasil); } } }}
```

# Break dan Continue pada Perulangan

**break:** Menghentikan perulangan dan keluar dari loop.

```
for (int i = 0; i < 10; i++) {  if (i == 5) {
break; // Keluar dari loop  }  print(i);}//
Output: 0, 1, 2, 3, 4
```

**continue:** Melewati iterasi saat ini dan melanjutkan ke iterasi berikutnya.

```
for (int i = 0; i < 5; i++) { if (i == 2) {
continue; // Lewati iterasi ini } print(i);}//
Output: 0, 1, 3, 4
```

# Perulangan Bersarang (Nested Loops)

Perulangan bersarang adalah perulangan di dalam perulangan. Berguna untuk bekerja dengan data multi-dimensi atau membuat pola.

```
// Membuat pola segitigafor (int i = 1; i <= 5; i++) {  String baris = '';  for (int j = 1; j <= i; j++) {    baris += '* ';  }
print(baris);}// Output:// *// * *// * * *// * * * *// * * * * *
```

Loop luar dieksekusi sekali untuk setiap baris, sedangkan loop dalam dieksekusi beberapa kali untuk setiap kolom dalam baris tersebut.

Perhatikan: Perulangan bersarang bisa memperlambat program jika jumlah iterasi sangat besar.

## OPERASI GIT LANJUTAN

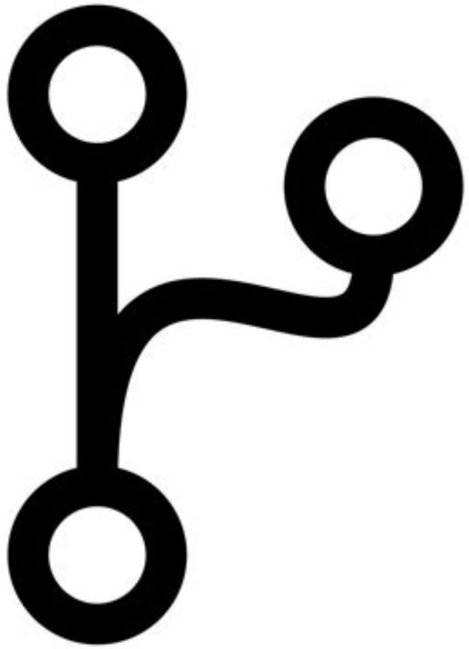
# Memperdalam Pengetahuan Git

# Pengantar Git Lanjutan

Git adalah sistem kontrol versi terdistribusi yang memungkinkan developer untuk:

- Melacak perubahan pada kode sumber
- Berkolaborasi dengan developer lain
- Mengelola versi berbeda dari aplikasi
- Kembali ke versi sebelumnya jika terjadi masalah

Pada pertemuan ini, kita akan mempelajari operasi Git lanjutan yang penting untuk pengembangan aplikasi mobile secara kolaboratif.



# Konsep Penting dalam Git

### Repository (Repo)

Penyimpanan proyek Git yang berisi semua file dan riwayat perubahan.

**Lokal:** Di komputer Anda

**Remote:** Di server (GitHub, GitLab)

### Branch

Jalur pengembangan terpisah yang memungkinkan pengerjaan fitur secara paralel tanpa mengganggu kode utama.

### Merge

Menggabungkan perubahan dari satu branch ke branch lain, biasanya dari branch fitur ke branch utama.

Memahami konsep ini penting untuk menggunakan Git secara efektif dalam proyek pengembangan aplikasi.

# Alur Kerja Dasar Git

Alur kerja dasar Git melibatkan tiga area utama:

**Working Directory:** Tempat Anda bekerja dan mengubah file

**Staging Area (Index):** Area persiapan untuk commit, file yang telah di-add

**Repository:** Database yang menyimpan semua perubahan yang telah di-commit

Memahami alur ini penting untuk menggunakan Git dengan benar.

# Git Clone: Mengunduh Repository

**git clone** digunakan untuk mengunduh seluruh repository dari server remote ke komputer lokal.

```
git clone https://github.com/user/repo-nama.git
```

Setelah clone, Anda akan memiliki:

- Salinan lengkap repository
- Semua riwayat commit
- Semua branch

Clone hanya perlu dilakukan **sekali** di awal. Setelah itu, gunakan git pull untuk memperbarui repository lokal Anda.

# Git Pull: Memperbarui Repository Lokal

**git pull** digunakan untuk mengambil perubahan terbaru dari repository remote dan menggabungkannya ke repository lokal Anda.

```
git pull origin main
```

git pull sebenarnya adalah kombinasi dari dua perintah:

**git fetch:** Mengambil perubahan dari remote tanpa menggabungkannya

**git merge:** Menggabungkan perubahan yang diambil ke branch lokal Anda

Selalu lakukan git pull sebelum mulai bekerja untuk memastikan Anda memiliki versi terbaru dari kode.

# Git Add dan Commit: Menyimpan Perubahan

**git add** memindahkan perubahan dari working directory ke staging area.

```
git add file.dart       # Menambahkan file tertentugit add
.               # Menambahkan semua perubahan
```

**git commit** menyimpan perubahan dari staging area ke repository lokal.

```
git commit -m "Pesan commit yang deskriptif"
```

Pesan commit sebaiknya deskriptif dan menjelaskan apa yang diubah, bukan bagaimana mengubahnya.

**Tips:** Commit secara teratur dengan perubahan kecil dan terfokus, bukan perubahan besar yang sulit dilacak.

# Git Push: Mengunggah Perubahan

**git push** mengunggah perubahan dari repository lokal ke repository remote.

```
git push origin main
```

Dimana:

**origin** adalah nama default untuk repository remote

**main** adalah nama branch yang ingin diunggah

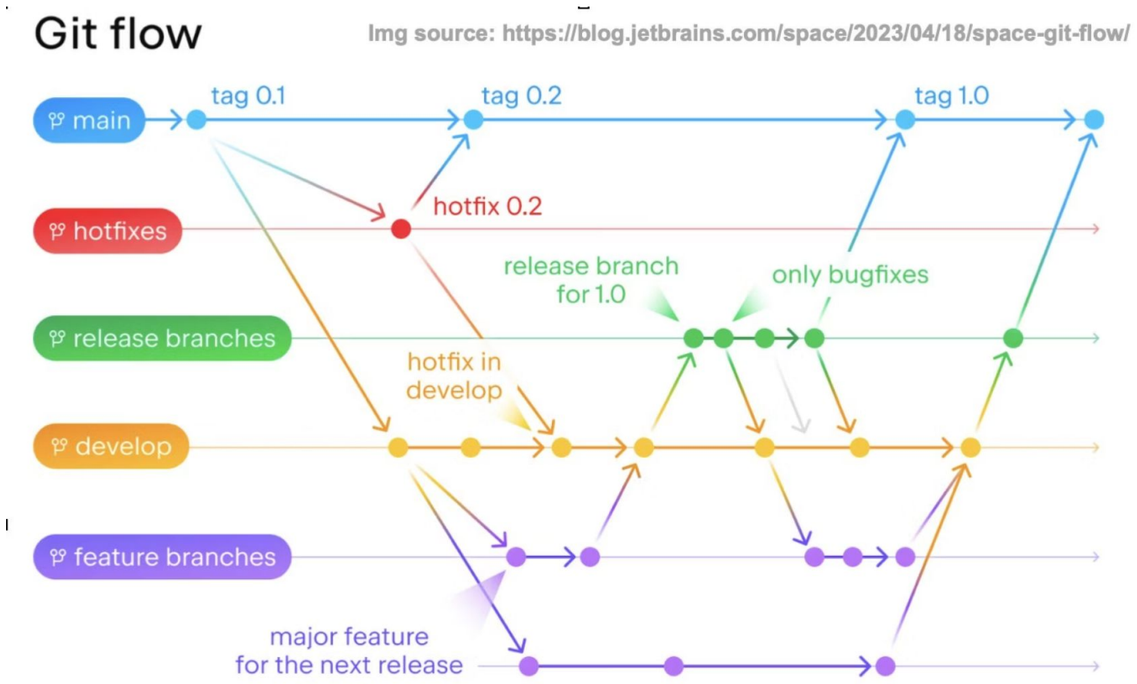
Pastikan untuk melakukan git pull terlebih dahulu untuk menghindari konflik jika bekerja dalam tim.

# Git Branch: Membuat Cabang Pengembangan

**git branch** digunakan untuk membuat, melihat, dan menghapus branch.

```
git branch                    # Melihat semua branchgit branch
nama-branch      # Membuat branch barugit branch -d nama-branch   #
Menghapus branch
```

Branch memungkinkan pengembangan fitur tanpa mengganggu kode utama.




- The `main` branch is for production code only.
- The `develop` branch is for development code.
- `feature` branches are created from the `develop` branch.
- `hotfix` branches are created from the `main` branch.
- `release` branches are created from the `develop` branch.

Branch biasanya digunakan untuk:

- Pengembangan fitur baru

# Git Checkout: Berpindah Antar Branch

**git checkout** digunakan untuk berpindah antar branch atau commit.

```
git checkout nama-branch     # Pindah ke branch tertentugit checkout -b nama-branch  # Membuat dan
pindah ke branch baru
```

Pastikan untuk commit atau stash perubahan Anda sebelum checkout untuk menghindari kehilangan perubahan.

**Tips:** Gunakan git checkout -b untuk membuat dan langsung berpindah ke branch baru dalam satu perintah.

# Git Merge: Menggabungkan Branch

**git merge** menggabungkan perubahan dari satu branch ke branch lain.

```
# Pastikan berada di branch tujuan (misalnya main)git checkout main# Gabungkan perubahan dari branch fiturgit merge fitur-login
```

Setelah merge berhasil, branch fitur dapat dihapus jika sudah tidak diperlukan:

```
git branch -d fitur-login
```

**Catatan:** Merge bisa menghasilkan konflik jika perubahan pada kedua branch bertentangan. Konflik harus diselesaikan secara manual.

# Skenario Penggunaan Git Branching

### Mulai dari branch utama

```
git checkout maingit pull origin main    # Pastikan up-to-date
```

### Buat dan pindah ke branch fitur

```
git checkout -b fitur-login
```

### Kerjakan fitur dan commit perubahan

```
/* Lakukan perubahan pada kode */git add .git commit -m "Implementasi form login"
```

### Gabungkan kembali ke branch utama

```
git checkout maingit merge fitur-login
```

### Unggah perubahan ke remote

```
git push origin main
```

# Git Status dan Log: Melihat Informasi

**git status** menampilkan status working directory dan staging area.

```
git status
```

Informasi yang ditampilkan:

- Branch aktif saat ini
- File yang dimodifikasi (tidak di-stage)
- File yang di-stage (siap di-commit)
- File baru yang belum di-track

**git log** menampilkan riwayat commit.

```
git log                    # Riwayat lengkapgit
log --oneline        # Format ringkas
```

Informasi dalam log:

- ID commit (hash)
- Author (nama dan email)
- Tanggal dan waktu commit
- Pesan commit

# Git Diff: Melihat Perubahan

**git diff** menampilkan perbedaan antara working directory, staging area, dan commit terakhir.

```
git diff                    # Perbedaan antara working
directory dan staginggit diff --staged         # Perbedaan
antara staging dan commit terakhirgit diff HEAD
# Perbedaan antara working directory dan commit
terakhirgit diff branch1..branch2 # Perbedaan antara dua
branch
```

Output git diff menunjukkan baris yang ditambahkan (+) dan dihapus (-), sehingga memudahkan untuk melihat apa yang berubah.

# Menangani Konflik Merge

Konflik merge terjadi ketika dua branch mengubah bagian yang sama dari file yang sama.

Ketika terjadi konflik, Git akan menandai area konflik dalam file:

```
<<<<<<< HEADkode dari branch saat ini=======kode dari branch yang di-merge>>>>>>> fitur-branch
```

Untuk menyelesaikan konflik:

1. Edit file untuk menghapus penanda dan pilih kode yang benar
2. git add file untuk menandai konflik sebagai teratasi
3. git commit untuk menyelesaikan merge

Konflik adalah bagian normal dari kolaborasi. Komunikasi dengan tim penting untuk menyelesaikan konflik dengan benar.

# Konsep Fork dan Pull Request

### Fork Repository

Membuat salinan repository orang lain ke akun GitHub/GitLab Anda

### Clone ke Lokal

Mengunduh fork repository ke komputer lokal Anda

### Buat Branch

Membuat branch baru untuk perubahan Anda

### Commit Perubahan

Membuat dan push commit ke fork Anda

### Pull Request

Mengusulkan perubahan Anda ke repository asli

Model fork dan pull request umum digunakan untuk berkontribusi pada proyek open source atau ketika Anda tidak memiliki akses langsung ke repository.

## STUDI KASUS: PENGEMBANGAN APLIKASI MOBILE

# Mengintegrasikan Percabangan, Perulangan, dan Git

# Studi Kasus: Aplikasi Kalkulator BMI

Kita akan mengembangkan aplikasi kalkulator BMI sederhana menggunakan Dart dengan menerapkan percabangan dan perulangan.

**Fitur Aplikasi:**

- Input tinggi dan berat badan
- Kalkulasi BMI
- Kategorisasi hasil (kurus, normal, gemuk)
- Riwayat perhitungan (menggunakan perulangan)

**Alur Kerja Git:**

1. Buat repository baru di GitHub
2. Clone repository ke lokal
3. Buat branch untuk setiap fitur:
    - branch input-validation
    - branch bmi-calculation
    - branch history-feature
4. Merge branch ketika fitur selesai
5. Push ke repository remote

## Implementasi Kode: Kalkulator BMI

```
// main.dartvoid main() { List> riwayat = [];  // Simulasi beberapa input hitungBMI(170, 65, riwayat);
hitungBMI(165, 75, riwayat); hitungBMI(180, 85, riwayat);  // Tampilkan riwayat menggunakan perulangan
tampilkanRiwayat(riwayat);}void hitungBMI(double tinggiCm, double beratKg, List> riwayat) { // Konversi
tinggi dari cm ke m double tinggiM = tinggiCm / 100; // Hitung BMI double bmi = beratKg / (tinggiM *
tinggiM); // Tentukan kategori menggunakan percabangan String kategori; if (bmi < 18.5) { kategori =
"Kurus"; } else if (bmi < 25) { kategori = "Normal"; } else if (bmi < 30) { kategori = "Gemuk"; } else {
kategori = "Obesitas"; }  // Simpan hasil ke riwayat riwayat.add({ 'tinggi': tinggiCm, 'berat': beratKg,
'bmi': bmi, 'kategori': kategori });}
```

# Ringkasan dan Tugas

## Konsep yang Dipelajari

- Percabangan: if-else, switch-case
- Perulangan: for, while, do-while, for-in
- Operasi Git lanjutan: clone, pull, push, branch, merge

## Tugas Mandiri

1. Buat program Dart sederhana yang menggunakan kombinasi percabangan dan perulangan untuk menyelesaikan masalah. Contoh: program untuk menghitung faktorial, cek bilangan prima, atau game tebak angka.
2. Buat repository GitHub untuk program tersebut dengan minimal 3 branch dan 5 commit yang menunjukkan perkembangan fitur.

## Kriteria Penilaian

- Ketepatan penggunaan percabangan dan perulangan
- Kerapian dan efisiensi kode
- Penamaan branch dan pesan commit yang deskriptif
- Riwayat pengembangan yang jelas melalui commit

Deadline: Sebelum pertemuan berikutnya. Silakan ajukan pertanyaan jika ada hal yang belum jelas.